BİL377 - HAFTA 06

Sıralama ve Seçim Uygulamaları

Partition, Quicksort, Quickselect ve sıra istatistikleri

Tam liste mi, tek bir sıra mı?

Sınav notlarında sıra istatistiği

Notlar geliş sırasıyla tutuluyor: [90, 60, 85, 70, 80]

Öğrenci	Not	Büyükten küçüğe sıra
Ayşe	90	1
Bekir	60	5
Ceren	85	2
Deniz	70	4
Ece	80	3

Büyükten küçüğe: 90, 85, 80, 70, 60 → üçüncü not 80.

Başarı sırası ve sıfır tabanlı indeks

Küçükten büyüğe sıralı görünüm: [60, 70, 80, 85, 90]

İstenen	Hesap	k	Sonuç
En küçük	1 – 1	0	60
İkinci en küçük	2 – 1	1	70
Üçüncü en yüksek	n – r = 5 – 3	2	80
En yüksek	n – 1 = 5 – 1	4	90

r: büyükten küçüğe 1 tabanlı sıra. k = n – r: artan dizide 0 tabanlı indeks.

Eşit notlarda sıra tanımı

Yeni liste: [95, 80, 80, 60]. Eşit notlar ayrı kayıtlar.

Soru	Sıralı görünüm	Cevap
İkinci en yüksek kayıt	95, 80, 80, 60	80
Üçüncü en yüksek kayıt	95, 80, 80, 60	80
Üçüncü farklı not	95, 80, 60	60

Bu derste sıra, tekrarlar dahil eleman sırasıdır. “Farklı değer” başka bir sözleşmedir.

Algoritma seçimi ve çıktı türü

Aynı veri üzerinde farklı çıktı türleri farklı işlemler gerektirir.

Görev	Yaklaşım	Gerekçe
En yüksek tek not	Tek geçiş	Görülen en büyük değerin saklanması
Tam sıralama	Sıralama	Her konumun değeri gerekli
k indeksindeki değer	Seçim	Yalnızca hedef konumun belirlenmesi

Bir çıktı yeterliyken bütün listeyi sıralamak ek iş olabilir.

Sıralama ile sıra istatistiği


```
notlar = [90, 60, 85, 70, 80]
r = 3
k = len(notlar) - r
sirali = sorted(notlar)
print(sirali)
print(sirali[k])
print(notlar)
```

Beklenen çıktı

```
[60, 70, 80, 85, 90]
80
[90, 60, 85, 70, 80]
```

sorted yeni liste döndürür; özgün notlar listesi korunur.

Pivot: karşılaştırma için seçilen değer

Son elemanın pivot olduğu ilk parçalama: pivot = 80.

Başlangıç: [90, 60, 85, 70, 80]

Hedef: [80 ve altı] | 80 | [80'den büyükler]

Sonuç: [60, 70] | 80 | [90, 85]

Pivotun indeksi q = 2; sağ taraf henüz sıralı değil.

Pivotu seçmek ile bütün listeyi sıralamak aynı işlem değildir.

Parçalamanın değişkenleri

Sınırlar dahildir: ilk çağrı partition(a, 0, 4).

Değişken	Başlangıç	Anlamı
lo / hi	0 / 4	Çalışılan parçanın ilk / son indeksi
pivot	a[4] = 80	Karşılaştırma değeri
j	0, 1, 2, 3	Sıradaki incelenecek konum
s	0	Küçük/eşit bölgenin boş konumu
q	Dönüşte 2	Pivotun kesinleşen indeksi

range(lo, hi), hi indeksini içermez; pivot döngüde incelenmez.

Lomuto parçalama: Python kodu


```
def partition(a, lo, hi):
    pivot = a[hi]
    s = lo
    for j in range(lo, hi):
        if a[j] <= pivot:
            a[s], a[j] = a[j], a[s]
            s += 1
    a[s], a[hi] = a[hi], a[s]
    return s
```

Lomuto parçalama

Son eleman pivot.

Küçük/eşit değer sola taşınır.

Son takas pivotu kesin yerine getirir.

Bu fonksiyon a listesini değiştirir; yalnızca q indeksini döndürür.

Parçalama izi: ilk iki karşılaştırma

Başlangıç: a = [90, 60, 85, 70, 80], pivot = 80, s = 0.

j	Koşul	İşlem	İşlemden sonraki dizi
0	90 ≤ 80: hayır	Takas yok; s = 0	[90, 60, 85, 70, 80]
1	60 ≤ 80: evet	a[0] ↔️ a[1]; s = 1	[60, 90, 85, 70, 80]

60 sola geçti. 90 silinmedi; incelenmiş büyükler bölgesinde kaldı.

Son iki karşılaştırma ve pivot takası

Devam: a = [60, 90, 85, 70, 80], s = 1.

Adım	Koşul / işlem	s	Dizi
j = 2	85 ≤ 80: hayır	1	[60, 90, 85, 70, 80]
j = 3	70 ≤ 80: evet; a[1] ↔ a[3]	2	[60, 70, 85, 90, 80]
Döngü sonu	a[2] ↔ a[4]; pivot yerine	2	[60, 70, 80, 90, 85]

Toplam 4 değer karşılaştırması, 3 takas satırı. Dönüş: q = 2.

Parçalamanın doğruluğu: döngü değişmezi

Her turun başında dört bölgenin anlamı korunur.

Aralık	Özellik
a[lo:s]	Her değer ≤ pivot
a[s:j]	Her değer > pivot
a[j:hi]	Henüz incelenmedi
a[hi]	Pivot burada bekliyor

Döngü bittiğinde bilinmeyen bölge boşalır; son takas pivotu araya koyar.

Partition sonucunun garantisi

Bir partition çağrısının gerçek çıktısı:

[60, 70] | 80 | [90, 85]

q = 2: artan sırada 2 indeksindeki değer 80.

90 ile 85'in kendi aralarındaki sırası çözülmedi.

k = 2 için aranan değer 80'dir.

Sadece gerekli konum kesinleştiyse seçim problemi çözülmüştür.

Alıştırma: parçalama

a = [75, 90, 60, 80]; lo = 0, hi = 3

Pivot nedir? Döngü j için hangi değerleri alır?

Son dizi ve q ne olur? Kaç karşılaştırma yapılır?

İkinci en yüksek not için hangi k gerekir?

Parçalama sonucunda pivotun iki yanındaki değerler hangi koşulları sağlar?

Quicksort: iki alt problemin sıralanması


```
def quicksort_yerinde(a, lo, hi):
    if lo >= hi:
        return
    q = partition(a, lo, hi)
    quicksort_yerinde(a, lo, q - 1)
    quicksort_yerinde(a, q + 1, hi)
```

Aynı partition

q konumu kesinleşir.

Sol: lo … q−1
Sağ: q+1 … hi

Boş veya tek elemanlı parça taban durumdur.

Pivot alt çağrılara dahil değildir. Her alt parça ana parçadan küçüktür.

Girdiyi koruyan Quicksort sarmalayıcısı


```
def quicksort(dizi):
    a = dizi.copy()
    quicksort_yerinde(a, 0, len(a) - 1)
    return a
```

Kopyalama ve dönüş

a = dizi.copy()

Özgün liste korunur.

Dönen değer, sıralanmış çalışma listesidir.

Çekirdek yerinde çalışsa da bu sarmalayıcı Θ(n) kopya alanı kullanır.

Quicksort: tam sıralama izi

Her satır bir partition çağrısından sonraki durumu gösterir.

Parça	Pivot → q	Dizi	Karşılaştırma
[0, 4]	80 → 2	[60, 70, 80, 90, 85]	4
[0, 1]	70 → 1	[60, 70, 80, 90, 85]	1
[3, 4]	85 → 3	[60, 70, 80, 85, 90]	1

Toplam 4 + 1 + 1 = 6 değer karşılaştırması. Artık bütün liste sıralı.

Quicksort’un zaman bağıntısı

n elemanlık parçada pivotun solunda p, sağında n–p–1 eleman vardır.

Sol parça: p; sağ parça: n – p – 1 eleman

Parçalama: n – 1 değer karşılaştırması

$$T(n) = T(p) + T(n - p - 1) + \Theta(n)$$

Aynı kod, farklı pivotlarla farklı bölünmeler üretir.

–1: pivotun kendisi alt problemlere dahil değildir.

Dengeli Quicksort'un zaman maliyeti

Varsayım: her pivot, aktif parçayı yaklaşık iki eşit parçaya böler.

Bir düzeyde işlenen parçaların toplamı en fazla n.

Parça boyutları: $n \to n/2 \to n/4 \to \dots \to 1$

Yaklaşık $\log_2 n$ düzey oluşur.

Dengeli Quicksort: $\Theta(n \log n)$

Dengeli bölünme varsayımı altında $T(n) = \Theta(n \log n)$.

Son pivot ve artan liste: kötü bölünme

a = [1, 2, 3, 4, 5, 6, 7, 8]. Her pivot parçanın en büyüğü.

Aktif boyut	Pivot	Sol / sağ boyut	Karşılaştırma
8	8	7 / 0	7
7	7	6 / 0	6
6	6	5 / 0	5
…	…	…	…
2	2	1 / 0	1

Toplam: 7 + 6 + 5 + 4 + 3 + 2 + 1 = 28.

En kötü durumun karşılaştırma sayısı

Son pivotla oluşan dengesiz bölünmede karşılaştırmaların toplamı:

$$C(n) = (n-1) + (n-2) + \ldots + 1$$

İleri ve ters toplamların toplamı: $2C(n) = n(n-1)$

$$C(n) = n(n-1)/2 = (n^2 - n)/2$$

n iki kat olunca karşılaştırmalar yaklaşık dört kat olur.

$n = 8 \rightarrow 8 \times 7 / 2 = 28$. Baş terim n^2 olduğu için $\Theta(n^2)$.

Master Teoremi’nin uygulanma koşulu

Dengeli durum: $T(n) \approx 2T(n/2) + \Theta(n)$

Uç pivot: $T(n) = T(n-1) + \Theta(n)$

Hangi bağıntının alt problemi n/b biçimindedir?

Uç pivot bağıntısının açılımında hangi toplam oluşur?

Master Teoremi, aT(n/b) + f(n) biçimindeki bağıntılara uygulanır.

Quickselect: hedef aralığın daraltılması

partition çağrısı pivotu q indeksine yerleştirdi. Hedef indeks k.

Karşılaştırma	Karar	Dönüş / yeni sınır
k = q	Cevap pivot	Dönüş değeri a[q]
k < q	Cevap solda	hi = q − 1
k > q	Cevap sağda	lo = q + 1

Quicksort iki dalı çözer. Quickselect bir dalı seçer; diğerini eler.

Quickselect: Python kodu


```
def quickselect(dizi, k):
    if not 0 <= k < len(dizi):
        raise IndexError("Geçersiz k")
    a = dizi.copy()
    lo, hi = 0, len(a) - 1
    while lo < hi:
        q = partition(a, lo, hi)
        if k == q:
            return a[q]
        if k < q:
            hi = q - 1
        else:
            lo = q + 1
    return a[lo]
```

$0 \le k < n$

Fonksiyon girdiyi kopyalar.

Hedefi içeren aralık daralır.

Tek aday kaldığında dönüş değeri a[lo] olur.

k, çalışma listesinin küresel indeksidir; sağa giderken k değişmez.

Quickselect izi: k = 3

Aynı liste: [90, 60, 85, 70, 80]. Hedef k = 3; k sabit kalır.

lo / hi	Pivot / q	Karar	Çalışma listesi
0 / 4	80 / 2	3 > 2: sağ aralık	[60, 70, 80, 90, 85]
3 / 4	85 / 3	3 = 3: sonuç 85	[60, 70, 80, 85, 90]

Cevap 85. Karşılaştırma sayısı 4 + 1 = 5; sol parça tekrar işlenmedi.

Quickselect izi: k = 0

Aktif aralık daralır; listenin tamamı her tur yeniden taranmaz.

Aktif parça	Pivot / q	Yeni aralık	Karşılaştırma
[0, 4]	80 / 2	[0, 1]	4
[0, 1]	70 / 1	[0, 0]	1
[0, 0]	Parçalama yok	a[0] = 60 döner	0

İki partition çağrısı; 5 değer karşılaştırması. Tek aday için döngüye girilmez.

Küresel ve yerel sıra indeksleri

Algoritma aynı a listesinde lo ve hi sınırlarıyla çalışır.

İlk hedef: a[3] → k = 3

Sağ parça: a[3:5] → lo = 3, hi = 4

Bu aralıkta da hedef aynı küresel konum: k = 3

Yeni bir alt liste kopyalansaydı yerel indeks 0 olurdu.

Küresel indeks k sabit kalır. Yerel indeks, alt listenin başlangıcına bağlıdır.

Dengeli Quickselect’in zaman maliyeti

Varsayım: her adımda aktif adayların yaklaşık yarısı kalır.

$$T(n) \approx T(n/2) + c \cdot n$$

$$\text{Toplam iş} \approx c \cdot (n + n/2 + n/4 + \ldots)$$

Parantez içindeki toplam 2n’den küçüktür.

Dengeli Quickselect: $\Theta(n)$

Tek dal üzerindeki iş, aktif aralığın boyutuyla birlikte azalır.

Beklenen süre ve en kötü durum

Rastgelelik, algoritmanın pivot seçimindedir; girdi sabit olabilir.

Sürüm / koşul	Süre	Açıklama
Son pivot; keyfî girdi	En kötü $\Theta(n^2)$	Her seferinde uç pivot olabilir
Dengeli seçim modeli	$\Theta(n)$	Kalan parça yaklaşık yarıya iner
Rastgele pivot + üç bölge	Beklenen $\Theta(n)$	Rastgele seçimler üzerinden ortalama
Aynı rastgele sürüm	En kötü $\Theta(n^2)$	Kötü pivot zinciri mümkündür

Beklenen süre, her çalıştırmada doğrusal süre garantisi değildir.

Rastgele pivot seçimi


```
from random import Random

rng = Random(377)
lo, hi = 0, len(a) - 1
p = rng.randrange(lo, hi + 1)
a[p], a[hi] = a[hi], a[p]
q = partition(a, lo, hi)
```

Pivotun sona taşınması

a: çalışma listesi.

randrange(lo, hi+1), hi indeksini de kapsar.

377: deneyin rastgelelik tohumu.

İndeksler eşit olasılıkla seçilir. Aynı değerler yine aynı pivot değerini verir.

Eşit değerlerde iki bölgeli parçalama

a = [80, 80, 80, 80, 80, 80, 80, 80], hedef k = 0.

İki bölgeli partition	İşlem	Sonuç
Koşul a[j] ≤ 80	Her tur doğru	Bütün değerler sola gider
İlk q	7	Yalnızca son konum kesinleşir
Sonraki boyutlar	7, 6, 5, …, 2	Yedi partition çağrısı
Karşılaştırmalar	7 + 6 + … + 1	28

Rastgele seçilen her indeksin değeri 80’dir. İki bölgeli yapı yine dengesizdir.

Üç bölge: küçük, eşit, büyük


```
def partition3(a, lo, hi, pivot):
    lt, i, gt = lo, lo, hi
    while i <= gt:
        if a[i] < pivot:
            a[lt], a[i] = a[i], a[lt]
            lt, i = lt + 1, i + 1
        elif a[i] > pivot:
            a[i], a[gt] = a[gt], a[i]
            gt -= 1
        else:
            i += 1
    return lt, gt
```

Dönüş: lt, gt

a[lo:lt] < pivot

a[lt:gt+1] = pivot

a[gt+1:hi+1] > pivot

lt ve gt: tamamlanmış eşitler bloğunun dahil sınırları.

Üç bölgenin sayısal izi

a = [80, 60, 80, 95, 80], pivot = 80. Satırlar işlemden sonrasını gösterir.

Değer / karar	lt / i / gt	Dizi
80 / eşit	0 / 1 / 4	[80, 60, 80, 95, 80]
60 / küçük	1 / 2 / 4	[60, 80, 80, 95, 80]
80 / eşit	1 / 3 / 4	[60, 80, 80, 95, 80]
95 / büyük	1 / 3 / 3	[60, 80, 80, 80, 95]
80 / eşit	1 / 4 / 3	[60, 80, 80, 80, 95]

lt = 1, gt = 3. k = 1, 2 veya 3 için cevap hemen 80.

Üç bölgeli parçalamada indeks güncellemesi

Örnek: i = 3, gt = 4 iken a[i] = 95 ve a[gt] = 80.

Takas: 95 sağa gider, 80 i konumuna gelir.

95 büyükler bölgesine geçer. gt bir azalır.

Yeni gelen 80 henüz incelenmediği için i aynı kalır.

i'nin hemen artması, bilinmeyen elemanın atlanmasına yol açar.

Küçük değerde gelen eleman bilinen eşit bölgesindendir; orada i ilerleyebilir.

Rastgele pivotlu üç bölgeli seçim


```
def quickselect3(dizi, k, seed=None):
    if not 0 <= k < len(dizi):
        raise IndexError("Geçersiz k")
    a = dizi.copy()
    rng = Random(seed)
    lo, hi = 0, len(a) - 1
    while lo < hi:
        pivot = a[rng.randrange(lo, hi + 1)]
        lt, gt = partition3(a, lo, hi, pivot)
        if k < lt:
            hi = lt - 1
        elif k > gt:
            lo = gt + 1
        else:
            return a[k]
    return a[lo]
```

Eşit blok tek adımda

k < lt → sol

k > gt → sağ

lt ≤ k ≤ gt → cevap pivot

Üreteç bir kez kurulur. Her turda aktif aralıktan rastgele pivot değeri seçilir.

Eşit değerlerde işlem sayılarının karşılaştırması

Sekiz eşit değer, k = 0. Çağrı, ziyaret ve karşılaştırma sayıları:

Ölçüt	İki bölge, son pivot	Üç bölge
Partition çağrısı	7	1
İncelenen değer ziyareti	28	8
Değer karşılaştırması	28 adet ≤	16 adet < veya >
Eşit blok	Her tur tek konum	[0, 7] birlikte tamam

Üç bölge kodunda eşit değerde iki karşılaştırma var; 8 ziyaret ≠ 8 karşılaştırma.

Algoritmaların ek bellek kullanımı

Yerinde çalışan çekirdek ile kullanıcı fonksiyonu ayrı değerlendirilir.

Kod	Ek alan	Neden
partition / partition3	Θ(1)	Sabit sayıda indeks ve değer
quickselect / quickselect3	Θ(n)	a = dizi.copy(); döngü yığını sabit
quicksort_yerinde	Dengeli Θ(log n) yığın	Her düzey bir çağrı
quicksort sarmalayıcısı	Θ(n) toplam ek alan	Kopya + en çok doğrusal yığın

quickselect için kopyayı kaldırmak alanı azaltır; o zaman girdi değişir.

Kararlılık: eşit notların sırası


```
kayitlar = [('Ayşe', 80),
            ('Bekir', 80),
            ('Ceren', 60)]
sonuc = sorted(kayitlar,
               key=lambda x: x[1])
print(sonuc)
```

Eşitler korunur

Önce Ceren 60.

Ayşe 80, Bekir 80'den önce kalır.

Ölçüt sadece nottur.

Python sorted kararlıdır. Bu takaslı Quicksort uygulaması kararlılık garantisi vermez.

Tek ve çift uzunluklu listelerde medyan

Tek uzunlukta orta değer, çift uzunlukta iki orta değerin ortalaması alınır.

Veri	Orta indeks(ler)	Medyan
[60, 70, 80, 85, 90]	n=5 → 2	80
[60, 70, 80, 90]	n=4 → 1 ve 2	(70 + 80)/2 = 75
[80, 80, 95]	n=3 → 1	80
[]	Orta konum yok	Bu sözleşmede ValueError

Çift n için medyan listede bulunan bir değer olmak zorunda değildir.

Seçim ile medyan hesabı


```
def medyan(dizi):
    n = len(dizi)
    if n == 0:
        raise ValueError("Boş liste")
    sag = quickselect3(dizi, n // 2)
    if n % 2:
        return sag
    sol = quickselect3(dizi, n // 2 - 1)
    return (sol + sag) / 2
```

Çift n: iki seçim

n//2: sağ orta.

n//2−1: sol orta.

Sabit iki doğrusal iş yine doğrusal ölçektedir.

Bu uygulama girdiyi korur; beklenen süre Θ(n), ek alan Θ(n).

Seçim fonksiyonlarının sınır durumları

Fonksiyon sözleşmesi, dönüş değerini, yan etkileri ve geçerli indeksleri kapsar.

Girdi / hedef	Beklenen	Kontrol edilen özellik
Not listesi, k=2	80	Normal durum
[80, 80, 95], k=1	80	Tekrarlı değer
[42], k=0	42	Tek aday, partition yok
Boş liste veya sınır dışı k	IndexError	Geçersiz hedefler
Çağrıdan sonra özgün liste	İlk hâliyle aynı	Girdinin korunması

Her geçerli k için dönüş değeri sorted(girdi)[k] değerine eşittir.

Girdi boyutu ve karşılaştırma sayısı

Artan liste + son pivot + k=0. Sayaç: yalnızca ≤ değer karşılaştırmaları.

n	Ölçülen karşılaştırma	n(n–1)/2
4	6	6
8	28	28
16	120	120
32	496	496

Son pivot ve k = 0 için kesin sayım: C(n) = n(n–1)/2.

Alıştırmalar: seçim ve medyan

Not listesi, k=2 → kaç partition ve kaç karşılaştırma?

Not listesi, k=3 → cevap ve aktif ikinci aralık?

Sekiz eşit değer, üç bölge → eşit blok hangi sınırlar?

[60, 70, 80, 90] → medyan neden 80 değil?

Sorulardaki not listesi: [90, 60, 85, 70, 80].

Örnek programın komut satırı seçenekleri


```
python3 kod/ders_ornekleri.py --ornek partition
python3 kod/ders_ornekleri.py --ornek secim --k 3
python3 kod/ders_ornekleri.py --ornek siralama
python3 kod/ders_ornekleri.py --ornek uc-bolge
python3 kod/ders_ornekleri.py --ornek tekrar
python3 kod/ders_ornekleri.py --ornek medyan
```

Program arayüzü

Her seçenek tek örnek çalıştırır.

--k: artan sırada hedef indeks.

Çalışma dizini: 6. hafta klasörü.

Program: kod/ders_ornekleri.py. Yalnızca Python standart kütüphanesini kullanır.

İhtiyaca göre algoritma seçimi

Araç seçimini çıktı türü, tekrarlar, kararlılık ve bellek gereksinimi belirler.

İhtiyaç	Araç	Özellik
Tek minimum / maksimum	min / max	Bir tarama yeterli
Tam ve kararlı sıralama	sorted(..., key=...)	Yeni liste, eşitlerde kararlılık
Tek sıra istatistiği	Quickselect	Yalnızca hedef sıranın değeri
Yoğun eşit değerler	Üç bölgeli seçim	Eşit blok birlikte tamamlanır
Girdinin korunması	Kopya alan sürüm	Θ(n) ek alan

Sıralama bütün konumları belirler. Seçim, hedef sıra istatistiğini bulur.

Ödev: tam k bursiyer seçimi

Kontenjan tam k kişidir. Eşit notlarda giriş sırası önceliklidir.

Girdi	Kural	Çıktı
Ayşe 95, Bekir 80, Ceren 80, Deniz 60	k = 2; notu yüksek olan önce seçilir	Ayşe ve Bekir
Eşit eşik notu	Girişte önce gelen öncelikli	Ceren dışarıda
Sonuç listesi	Seçilenler giriş sırasıyla döner	Tam 2 kayıt

Yöntem: eşik ve tarama. Sınır durumları: k=0, k=n ve geçersiz k.

Değerlendirme soruları

Partition neden bütün listeyi sıralamak zorunda değil?

Quicksort ile Quickselect'in bağıntıları neden farklı?

Rastgele pivot, eşit değer sorununu tek başına çözer mi?

Kopya alan Quickselect neden Θ(1) alanlı sayılmaz?

Bir sonraki hafta: tekrar hesaplanan alt problemler ve dinamik programlama.